

PLATAFORMA DE TELEMEDICINA · MÓDULO 5 · VERSÃO 1.0

Login e as Primeiras Telas

O site deixa de ser um painel de luzes e vira o começo do produto: criar conta, entrar, dizer quem você é (paciente ou médico) e ver o próprio perfil — tudo em cima do que o Módulo 4 já provou pelo terminal. O painel de luzes continua existindo, em /diagnostico, só para quem está logado.

O que este módulo pressupõe

O Módulo 4 terminou: cinco luzes verdes, `pnpm api:testar-fluxo` em “Tudo certo”, commit enviado.

O projeto que acompanha este PDF é `telemedicina-modulo-05.zip`. Testado do zero: instalação, tipos nos 4 pacotes, 17 testes, `next build` do site sem erro, e as telas exercitadas no servidor com três situações — sem login, logado sem perfil, logado com perfil — contra a API e o banco reais (o Supabase Auth foi simulado, porque só você tem o seu projeto; a parte que fala com ele é a mesma que o `testar-fluxo` do Módulo 4 já provou de verdade).

0. Onde você vai chegar

Ao terminar, você abre `http://localhost:3000`, cria uma conta com seu e-mail, diz que é paciente (ou médico, com CRM), e cai na tela **Início** com seu nome, um carimbo com o papel e a lista de médicos. Sai, entra de novo, e está tudo lá. No Supabase, a linha `perfil.criado` da auditoria tem o seu IP.

Tela	Login	O que faz
/	não	Capa. Quem já está logado nem vê: vai para /inicio.
/criar-conta	não	E-mail e senha → Supabase Auth. Com sessão na mão, vai para /completar-perfil.
/entrar	não	Login. ?depois=/x volta para onde a pessoa queria ir.
/completar-perfil	sim	Paciente ou médico, nome, e o bloco de cada um → POST /perfis. Quem já tem perfil é mandado para /inicio.
/inicio	sim	Meu perfil (GET /perfis/eu) e a lista de médicos (GET /medicos). Sem perfil → /completar-perfil.
/diagnostico	sim	O painel de luzes dos Módulos 2 a 4, intacto.

Nenhuma variável de ambiente nova. O `pnpm verificar` ganha o passo [8/8], que confere as telas e — o mais importante — que o CORS da API aceita o site.

1. Os conceitos

Palavra	O que é (versão de 10 anos)	No nosso projeto
Sessão	O “já mostrei o crachá, pode me deixar entrar nas próximas portas”. Dura 1 hora e é renovada sem a pessoa perceber.	Guardada num cookie chamado <code>sb-...-auth-token</code> , gravado pelo cliente do navegador.
Cookie	Um bilhete que o navegador manda junto com cada pedido ao site. O servidor lê e sabe quem está pedindo.	É como o Server Component de /inicio sabe quem você é antes de montar a página.
Middleware	Um porteiro que roda antes de toda página. Olha o bilhete e decide: entra, ou vai para /entrar.	<code>apps/web/src/middleware.ts</code>
Server Component	Página montada no servidor; o navegador recebe HTML pronto. Não vê token, não vê a API.	/inicio, /completar-perfil (a moldura), /diagnostico, o layout.
Client Component	Pedaço que roda no navegador porque precisa reagir a digitação e cliques. Começa com “use client”.	Os três formulários e o botão Sair.
CORS	A regra do navegador: um site só pode chamar uma API que o aceitou explicitamente.	<code>ORIGEM_PERMITIDA=http://localhost:3000</code> em <code>apps/api/.env</code> é isso.

Quem fala com quem (ADR-0007)

Navegador → **Supabase Auth**: só para criar conta, entrar e sair. Com a chave pública.

Navegador → **API**: os formulários (POST /perfis), com o token no cabeçalho. Nunca direto no banco.

Servidor do site → **API**: as telas montadas no servidor (GET /perfis/eu, GET /medicos), com o token lido do cookie.

API → **banco**: a única que escreve, com a chave secreta e auditoria. O site não tem a chave secreta e não tem DATABASE_URL.

2. O que há de novo no projeto

```
apps/web/src/
|-- middleware.ts          renova a sessao e faz a portaria das paginas
|-- lib/
|   |-- supabase-servidor.ts  cliente do Supabase que le o cookie (servidor)
|   |-- supabase-navegador.ts (ja existia) cliente que grava o cookie (navegador)
|   |-- api.ts               chamarApi(): a unica forma de falar com apps/api
|   |-- validacao.ts         CPF, data de nascimento, senha - antes de ir a rede
|   |-- validacao.test.ts    4 testes
|-- componentes/
|   |-- Campo.tsx Botao.tsx Aviso.tsx      blocos de formulario, acessiveis
|   |-- Carimbo.tsx          o carimbo do receituario (papel, CRM, CPF)
|   |-- BotaoSair.tsx        signOut + volta para a capa
`-- app/
    |-- layout.tsx           cabecalho muda conforme logado ou nao
    |-- page.tsx             capa (era o painel de luzes)
    |-- entrar/              page.tsx + FormularioEntrar.tsx
    |-- criar-conta/         page.tsx + FormularioCriarConta.tsx
    |-- completar-perfil/    page.tsx + FormularioPerfil.tsx
    |-- inicio/page.tsx      meu perfil + medicos
    |-- diagnostico/page.tsx o painel de luzes, agora aqui
docs/adr-0007-sessao-em-cookie-e-site-sem-escrita.md
scripts/verificar-ambiente.mjs passo [8/8]
```

A identidade visual está em `globals.css`: papel claro e frio, tinta azul-marinho de caneta, e o verde-azulado do tubo do estetoscópio como única cor de destaque. Título em serifa (a fonte do receituário), corpo em sans, e monoespaçada só para números que certificam (CRM, CPF). O **carimbo** — borda dupla, leve inclinação — é a assinatura do site e aparece só em dados de identidade. Se você o vir em outro lugar, está errado.

Dependências: `@supabase/ssr` já estava declarado no Módulo 2 e agora é usado de verdade; `vitest` entra em `apps/web`. Rode `pnpm install` antes de qualquer coisa.

3. O fluxo, passo a passo por dentro

3.1 Criar conta

`FormularioCriarConta` valida a senha no navegador (8+ caracteres, letras e números; `problemaDaSenha()`), chama `supabase.auth.signUp` e olha a resposta. Dois caminhos, conforme a configuração do seu projeto no Supabase:

- **Confirm email desligado** (recomendado em dev, seção 4): a resposta já traz a sessão; o cookie é gravado e a pessoa vai para `/completar-perfil`.
- **Confirm email ligado**: sem sessão; a tela mostra “Enviamos um link para ...”. A pessoa clica no e-mail, volta em `/entrar`. (Em produção é assim; Módulo 8.)

3.2 Entrar

`signInWithPassword` → cookie gravado → `router.push(destino) + router.refresh()`. O `refresh` é o que faz o layout (Server Component) reler o cookie e trocar “Entrar / Criar conta” por “Início / Sair”. Os erros do Supabase são traduzidos: “Invalid login credentials” vira “E-mail ou senha incorretos”.

3.3 A portaria (middleware)

Em cada requisição: monta um cliente Supabase com os cookies do pedido, chama `getUser()` (que valida o token no Supabase e, se estiver perto de vencer, renova e regrava o cookie), e decide. Sem usuário em `/inicio`, `/completar-perfil` ou `/diagnostico` → `/entrar?depois=...`. Com usuário em `/entrar` ou `/criar-conta` → `/inicio`. É o único lugar do site que grava cookie de sessão pelo servidor.

3.4 Completar perfil

A página (servidor) pergunta à API `GET /perfis/eu` com o token do cookie: se veio 200, a pessoa já tem perfil e é mandada para `/inicio`; se 404, mostra o formulário. O formulário (navegador) valida campo a campo — CPF com os dígitos verificadores de verdade, data no passado, CRM de 4 a 7 dígitos — pega o token da sessão e faz `POST`

/perfis. Um 400 da API vem com detalhes: [{campo, problema}] e cada problema aparece no campo certo; 409 diz que a conta já tem perfil.

3.5 Início

Server Component puro: token do cookie → GET /perfis/eu e GET /medicos → HTML. O navegador recebe só a página pronta; não há JavaScript que conheça o token nesta tela. 404 em /perfis/eu redireciona para /completar-perfil — é assim que quem fechou a aba no meio do cadastro retoma de onde parou.

4. Preparar o projeto no Supabase (clique a clique)

Duas configurações do Auth para o curso rodar sem depender de e-mail chegando.

1. Painel do Supabase > projeto telemedicina-dev > menu lateral **Authentication** > **Sign In / Providers** (em projetos mais antigos, **Providers**).
2. Clique em **Email**. Deixe **Enable sign in with email** ligado. **Desligue** “Confirm email”. Clique em **Save**.
3. Ainda em Authentication, entre em **URL Configuration**. Em **Site URL** coloque `http://localhost:3000` e clique em **Save**. (É para onde os links de e-mail voltam quando a confirmação estiver ligada; em dev não é usado, mas deixa o projeto coerente.)

Por que desligar a confirmação em dev — e religar em produção

Com a confirmação ligada, cada conta nova depende de um e-mail chegar e ser clicado, e o SMTP padrão do Supabase entrega poucos por hora. Para aprender e testar, isso só atrapalha. Em produção (Módulo 8) ela volta a ficar ligada, com SMTP próprio — e o código deste módulo já trata os dois casos, então nada precisa mudar no site.

5. Atualizar o projeto

Ordem obrigatória

1) Ctrl+C no `pnpm dev`. 2) Extrair o zip por cima. 3) `pnpm install` antes de qualquer outro comando.

```
pnpm install
pnpm typecheck
pnpm test
pnpm verificar
```

O que você deve ver

`pnpm test`: quatro arquivos — 3 (db), 5 + 5 (api), 4 (web). Total 17.

`pnpm verificar` termina com o passo [8/8] todo ok e “Tudo certo. Proximo passo: `pnpm dev` e abra `http://localhost:3000` para criar sua conta”.

Se o [8/8] acusar `ORIGEM_PERMITIDA`: em `apps/api/.env` ela tem de ser exatamente `http://localhost:3000`, senão o navegador bloqueia o `POST /perfis` do formulário (erro de CORS no console).

6. Provar, agora com telas

```
pnpm dev
```

1. Abra `http://localhost:3000`. É a capa: título, dois botões. O cabeçalho mostra **Entrar** e **Criar conta**.
2. Clique em **Criar conta**. Digite seu e-mail e uma senha fraca de propósito (“abc”). Clique em **Criar conta**: o campo Senha mostra “Use ao menos 8 caracteres.” sem chamar a rede. Corrija (ex.: `TeLemed2026`) e envie.
3. Você cai em **Quem é você?** e o cabeçalho já diz **Início · Diagnóstico · Sair** — a sessão está no cookie. Escolha **Paciente**, nome completo, data de nascimento. No CPF, digite `111.111.111-11`: ao salvar, “Este CPF não é válido”. Apague o CPF (é opcional) e clique em **Salvar perfil**.
4. Tela **Início**: seu nome em serifa, o carimbo **PACIENTE**, data de nascimento, e “Nenhum médico cadastrado ainda”.
5. Aperte F5. Continua tudo lá (sessão no cookie). Clique em **Diagnóstico**: as cinco luzes, agora dentro do site logado.
6. Clique em **Sair**. Volta à capa, cabeçalho com Entrar/Criar conta. Digite `http://localhost:3000/inicio` na barra: vai para `/entrar?depois=%2Finicio`. Entre com o e-mail e a senha: cai direto em `/inicio`.
7. Segunda conta, agora médica: **Sair**, **Criar conta** com outro e-mail (pode ser `seu+medica@...` se o seu provedor aceitar), **Médico(a)**, CRM 123456, UF SP, especialidade “Clínica geral”, salvar. **Início** mostra dois

carimbos: MEDICO e CRM 123456-SP, e “Colegas na plataforma” com você mesma.

8. Saia, entre com a conta de paciente: “Medicos disponiveis” lista a médica com o CRM.

O que aconteceu no banco enquanto isso

SQL Editor: select quando, acao, detalhes, ip from auditoria order by id desc limit 5; — duas linhas perfil.criado, uma com {"papel": "paciente"} e outra {"papel": "medico"}, com o IP do seu computador. Nenhuma tela escreveu no banco: as duas passaram pela API.

Se aparecer...	Causa	Faça
Ao salvar o perfil: “A API em http://localhost:3333 nao respondeu”	A API não está no ar	Terminal: apps/api dev: — pnpm install feito? porta 3333 livre?
Ao salvar o perfil: nada acontece e o console do navegador (F12) fala em CORS	ORIGEM_PERMITIDA da API não é http://localhost:3000	Corrija em apps/api/.env e reinicie o pnpm dev. O [8/8] do verificar acusa isso.
Criar conta mostra “Enviamos um link para...”	Confirm email está ligado	Seção 4, ou abra o link do e-mail e entre por /entrar.
Criar conta: “Este e-mail ja tem conta”	Você já criou antes (talvez num testar-fluxo interrompido)	Use Entrar; ou Authentication > Users > apague.
Depois de entrar, cabeçalho continua “Entrar / Criar conta”	O cookie não foi gravado — NEXT_PUBLIC_SUPABASE_URL diferente entre .env.local e o projeto onde a conta existe	pnpm verificar; as duas URLs (API e site) têm de ser do mesmo projeto.
/inicio mostra “Voce ainda nao tem perfil” em vermelho	A API respondeu algo diferente de 200 e 404 (ex.: 401 por token de outro projeto)	Saia, entre de novo. Se persistir, terminal da API.

7. O que cada arquivo novo faz

Arquivo	O que faz
src/middleware.ts	createServerClient com os cookies do pedido; getUser(); redireciona conforme PAGINAS_PROTEGIDAS e PAGINAS_SO_DESLOGADO; matcher exclui estáticos. Lê process.env direto porque roda no edge.
src/lib/supabase-servidor.ts	criarClienteServidor() com cookies() do Next; usuarioAtual() (getUser) e tokenAtual() (access_token da sessão) para as páginas de servidor.
src/lib/api.ts	chamarApi(rota, {metodo, token, corpo}) → {status, ...envelope}. Nunca lança por status; timeout de 8 s; sem cache.
src/lib/validacao.ts	soDigitos, cpfValido (algoritmo da Receita), formatarCpf, dataNascimentoValida, problemaDaSenha. Só para feedback imediato: a API valida de novo.
src/componentes/Campo.tsx	label + input + ajuda + erro, ligados por aria-describedby e aria-invalid; erro em texto, com role=alert.
src/componentes/Carimbo.tsx	A classe .carimbo do globals.css. Só para papel, CRM e CPF.
src/app/layout.tsx	Server Component: usuarioAtual() decide o cabeçalho. Link “Pular para o conteudo” para teclado e leitor de tela.
src/app/entrar/*	Página fina (lê ?depois) + FormularioEntrar (signInWithPassword, tradução de erros, push + refresh).
src/app/criar-conta/*	Página + FormularioCriarConta (problemaDaSenha, signUp, os dois caminhos de confirmação).
src/app/completar-perfil/*	Página checa GET /perfis/eu (200 → /inicio); FormularioPerfil valida, monta o corpo conforme o papel, POST /perfis, distribui os detalhes do 400 por campo.
src/app/inicio/page.tsx	GET /perfis/eu (404 → /completar-perfil) e GET /medicos; carimbos; datas em pt-BR.

Arquivo	O que faz
src/app/diagnostico/page.tsx	O antigo page.tsx, com as classes da nova paleta.
scripts/verificar-ambiente.mjs	[8/8]: telas e middleware presentes; @supabase/ssr declarado; ORIGEM_PERMITIDA inclui o site; NEXT_PUBLIC_API_URL bate com PORT.
docs/adr-0007-...	Sessão em cookie, portaria no middleware, getUser() sempre, site nunca escreve no banco.

8. Git: commit do módulo

```
git status
git add .
git commit -m "feat(web): login, cadastro, completar perfil e tela inicial; painel em /diagnostico"
git push
```

9. Checklist do Módulo 5

- ☐ Confirm email desligado no telemedicina-dev; Site URL = http://localhost:3000
- ☐ Zip extraído por cima; pnpm install; typecheck limpo; pnpm test com 17 passed
- ☐ pnpm verificar em “Tudo certo” com o [8/8] ok
- ☐ Conta de paciente criada pelo site; senha fraca e CPF inválido recusados antes da rede
- ☐ Início mostra o carimbo do papel; F5 mantém a sessão; Sair e Entrar funcionam; ?depois= volta para onde você queria ir
- ☐ Conta de médica criada; Início mostra CRM; a paciente vê a médica na lista
- ☐ auditoria tem duas linhas perfil.criado com IP
- ☐ Você sabe explicar por que /inicio é Server Component e o formulário é Client Component
- ☐ Você sabe dizer por que o site não tem DATABASE_URL nem chave secreta e mesmo assim mostra seus dados
- ☐ Commit feito e enviado, sem nenhum .env

Próximo: Módulo 6 — Agenda e consultas. O médico define horários, o paciente escolhe um e marca (POST /consultas, com as regras do banco que o Módulo 3 já garante: fim depois do início, sem sobreposição — esta última entra como nova migração). Tela de agenda para os dois lados, status da consulta e auditoria de cada mudança. Mesmo padrão: cada tela com o que você deve ver, tudo testado antes de ser entregue.